

클라우드컴퓨팅

텀 프로젝트 1: 제안서

대표학생 이름	김성욱
대표학생 학번	202024191
대표학생 소속 학과/대학	정보의생명공학대학 정보컴퓨터공학부
분반	059

<주의사항>

- 팀 과제입니다. (팀 대표 또는 팀원 중 아무나 한 명만 제출하면 됩니다.)
- **각각의 질문 아래에 답을 작성 후 제출해 주세요.**
 - 소스코드/스크립트 등을 제출하는 경우, 해당 파일의 이름도 적어주세요.
- PLATO 제출 데드라인: **공지사항 참고**
 - 데드라인을 지나서 제출하면 0 점
 - 주말/휴일/학교행사 등 모든 날짜 카운트 함
 - 부정행위 적발 시, 원본(보여준 사람)과 복사본(베낀 사람) 모두 0 점 처리함
 - 예외 없음
- PLATO 에 아래의 파일을 제출 해 주세요
 - 보고서(**PDF 파일로 변환 후 제출, 또는 워드문서 그대로 제출**)
 - 보고서의 경우 PDF/DOC/DOCX 이외의 확장자는 제출 불가
 - 다른 확장자로 제출했는데 채점자 컴퓨터에서 파일이 열리지 않으면 0 점 처리
 - 보고서 파일명 및 보고서 첫 페이지에 이름과 학번을 입력 해 주세요.

<개요>

이번 과제는 팀 프로젝트 제안서를 제출하는 내용입니다.

<팀 프로젝트 : 제안서>

[1] 개발 내용 소개

팀 프로젝트 주제 및 팀 프로젝트에서 개발하고자 하는 시스템/서비스/SW 에 대해 소개하세요.

답변 1(프로젝트 주제): 아래에 한 줄로 프로젝트 주제를 정리

Container 기반 AI Agent 격리 및 권한 제어 시스템 개발

답변 2(프로젝트 소개): 아래에 한 문단으로 프로젝트를 소개

이 프로젝트는 인간의 업무 구조와 유사한 환경을 구현함으로써 Agent 의 행동을 안전하고 예측 가능하게 강제하는 격리 시스템을 개발한다. OpenClaw 등 Multi-Agent Orchestration Harness 에서 보통 Agent 가 동일한 권한으로 실행되어 서로의 파일과 디렉토리에 제한 없이 접근할 수 있으며 Prompt 수준의 제한만으로는 차단을 보장할 수 없다. 이 프로젝트에서는 이를 해결하기 위해 조직의 운영 방식을 모방한 구조를 구현한다. 조직에서 구성원이 타 부서 문서에 물리적으로 접근할 수 없듯이 각 Agent 는 자신의 Container 내 파일 시스템에만 접근할 수 있으며 다른 Agent 의 파일과 디렉토리는 OS 수준에서 차단된다. 그리고 부서 간 협업이 공식 결재를 통해 이루어지듯 Agent 간 통신 역시 사전에 정의된 Port 를 통해서만 허용되어 어떤 데이터가 어떤 경로로 오갔는지 추적 가능한 형태로 기록된다. 이처럼 Prompt 지시 대신 구조적으로 불가능한 환경을 제공함으로써 Agent 가 역할 범위를 벗어나는 문제를 Prompt 차원이 아닌 시스템 설계 차원에서 해결한다.

[출처: <https://openclaw.ai/>, <https://github.com/openclaw/openclaw>]

[2] 필요성 및 활용방안.

개발하고자 하는 SW 의 필요성에 대해 설명하세요 개발 결과물이 어떻게, 그리고 어떤 상황에서 사용될지(Use-Case) 설명하세요.

답변:

• 필요성

AI Agent 기반 자동화 시스템의 도입이 빠르게 확산되는 가운데, OpenClaw 는 개인 및 팀 단위 자동화 환경에서 널리 활용되고 있다. 그러나 OpenClaw 의 Agent 아키텍처는 내장된 보안 제약이 부재하여 심각한 권한 관리 취약점을 내포하고 있다.

이와 관련하여 실제 환경에서 다음과 같은 사례들이 보고됐다. Multi-agent 환경을 운용하던 사용자는 SOUL.md 에 특정 파일에 대한 접근 금지 지시를 명시했음에도 불구하고 일부 모델이 해당 지시를 무시하고 설정 파일을 무단으로 열람하는 현상을 확인했다. 더 주목할 만한 점은 동일한 Agent 가 사용자의 직접 요청에는 접근을 거부하면서도 다른 Agent 가 A2A(Agent-to-Agent) 프로토콜을 통해 동일한 요청을 전달하자 이를 수행했다는 것이다. 이는 프롬프트 수준의 접근 제한이 모델의 동작을 일관되게 강제하지 못하며 Agent 간 통신 경로에서 우회될 수 있음을 시사한다.

[출처: <https://github.com/openclaw/openclaw/issues/12202>]

또한 파일 시스템 접근 및 셸 실행 권한이 부여된 DevOps Agent 가 자신의 SOUL.md 를 주기적으로 읽고 새로운 명령을 추가하는 크론잡을 자율적으로 등록한 사례도 확인됐다. 운영자는 약 2 주가 경과한 후 Agent 의 행동 변화를 통해 해당 사실을 인지했다.

[출처: <https://www.crewclaw.com/blog/openclaw-agent-permissions-safety>]

이는 LLM 기반 Agent 의 행동이 운영자의 의도와 무관하게 자율적으로 변화할 수 있음을 보여주는 사례로, 자연어 지시만으로는 Agent 의 동작 범위를 신뢰성 있게 제한하기 어렵다는 점을 입증한다. 따라서 소프트웨어 아키텍처 수준에서 Agent 의 리소스 접근을 물리적으로 차단하는 격리 메커니즘의 설계가 요구된다.

• 활용방안 (Use-Case)

- 본 시스템은 다음과 같은 상황에서 활용된다.

- 1) Multi-agent 개발 자동화 환경: 개발 Agent, 테스트 Agent, 배포 Agent 가 동시에 동작할 때 각 Agent 가 자신의 작업 디렉토리 밖에 접근하지 못하도록 강제로 제한해야 하는 경우.

- 2) 기업 내 AI Agent 도입: 여러 부서의 Agent 가 동일한 인프라에서 동작할 때, 인사 Agent 가 재무 데이터에, 또는 외부 연동 Agent 가 내부 설정 파일에 접근하는 것을 원천 차단해야 하는 경우.
- 3) AI Agent 연구 및 실험 환경: 다양한 모델과 Agent 를 실험할 때, 한 Agent 의 오작동이 다른 Agent 나 호스트 시스템에 영향을 미치지 않도록 격리된 실행 환경을 제공해야 하는 경우.
- 4) 개인 사용자 자동화: 개인이 여러 AI Agent 를 동시에 운용할 때, 각 Agent 가 의도한 작업 범위 내에서만 동작하도록 보장하여 의도치 않은 파일 수정이나 삭제를 방지하는 경우.

[3] 선행기술 조사

개발하고자 하는 SW/시스템과 동일한 또는 유사한 기술이 이미 개발됐는지를 조사하세요. 관련 제품을 조사한 결과를 첨부하거나, 관련 논문/특허를 검색한 결과를 첨부해도 됩니다. 관련 제품/기술에 대한 간략한 소개 및 출처도 작성하세요.

답변:

- **유사 기술 조사**

- 1) NemoClaw (NVIDIA)

- OpenClaw의 보안 취약점을 해결하기 위해 NVIDIA가 개발한 엔터프라이즈 보안 레이어로, OpenClaw 위에 올라가는 런타임 레이어 형태로 동작한다. NVIDIA의 OpenShell 런타임을 통해 네트워크 및 파일 시스템 격리, 실시간 정책 승인, 완전한 로컬 추론을 지원한다. OpenShell의 핵심 설계 원칙은 Agent 프로세스 외부에 정책 집행 레이어를 두는 것으로 Agent 자신이 자신을 제한하는 제어를 우회할 수 없다는 점이 특징이다. 그러나 NemoClaw는 OpenClaw 전체를 하나의 Container로 감싸는 방식이므로 Container 내부에서 Agent 간 격리는 여전히 이루어지지 않으며 기존 OpenClaw 사용자들의 마이그레이션이 어렵다는 한계가 있다.

[출처: <https://github.com/NVIDIA/NemoClaw>, <https://developer.nvidia.com/blog/build-a-secure-always-on-local-ai-agent-with-nvidia-nemocl原因-and-openclaw/>]

- 2) AI Agent Sandboxing 동향

- 2026년 기준 AI Agent 보안 분야에서는 Docker/runc 기반의 공유 커널 Container 격리만으로는 비신뢰 코드 실행 환경에 충분하지 않다는 인식이 확산되고 있다. 특히 LLM이 생성하거나 사용자로부터 전달받은 코드는 잠재적으로 신뢰할 수 없는 것으로 간주되어야 하며 Multi-agent 환경에서는 한 Agent의 권한이 다른 Agent로 전파되는 Cross-agent Privilege Escalation (Agent 간의 권한 상승) 및 인접 Agent로의 권한 횡적 전파 위험이 주요 위협 모델로 논의되고 있다. 또한 최근에는 Agent 간 통신 세션을 악용하여 숨겨진 명령을 삽입하는 Agent Session Smuggling 공격 벡터도 제안되며 Agent 간의 신뢰 경계 보안의 중요성이 커지고 있다.

[출처: <https://manveerc.substack.com/p/ai-agent-sandboxing-guide>,
<https://unit42.paloaltonetworks.com/agent-session-smuggling-in-agent2agent-systems/>, <https://noma.security/resources/access-control-for-ai-agents/>]

- 관련 논문

- Shandong University (산둥대학) 연구팀(Shan et al., 2026)은 OpenClaw 의 보안 취약점을 체계적으로 분석한 논문 “Don’t Let the Claw Grip Your Hand: A Security Analysis and Defense Framework for OpenClaw”를 발표했다. 이 논문은 OpenClaw 가 내장된 보안 제약 없이 다양한 상용 LLM 과 연동될 수 있어 Agent 취약점 평가의 기준 모델로 적합하다고 분석하고, 악의적인 명령에 대한 OpenClaw 의 취약성을 두 단계에 걸쳐 체계적으로 평가했다.

[출처: <https://arxiv.org/pdf/2603.10387>]

[4] 제안하는 SW/시스템의 구성도

최종 결과물의 구성 및 동작방식을 다이어그램을 그려서 설명하세요. 예를 들어, [WAS]+[DB]로 구성된 SW 인 경우, [사용자], [WAS], [DB]를 나타내는 다이어그램을 각각 그리고 [사용자]가 [WAS]로 어떤 요청을 보내는지, [WAS]가 사용자에게 어떤 답변을 보내는지, [WAS]와 [DB]사이에 어떤 요청/응답이 오가는지를 설명하는 다이어그램을 그리세요. 그림에 텍스트를 충분히 추가하여 어떤 데이터/명령이 오고 가는지, 어떻게 상호작용 하는지를 나타내세요.

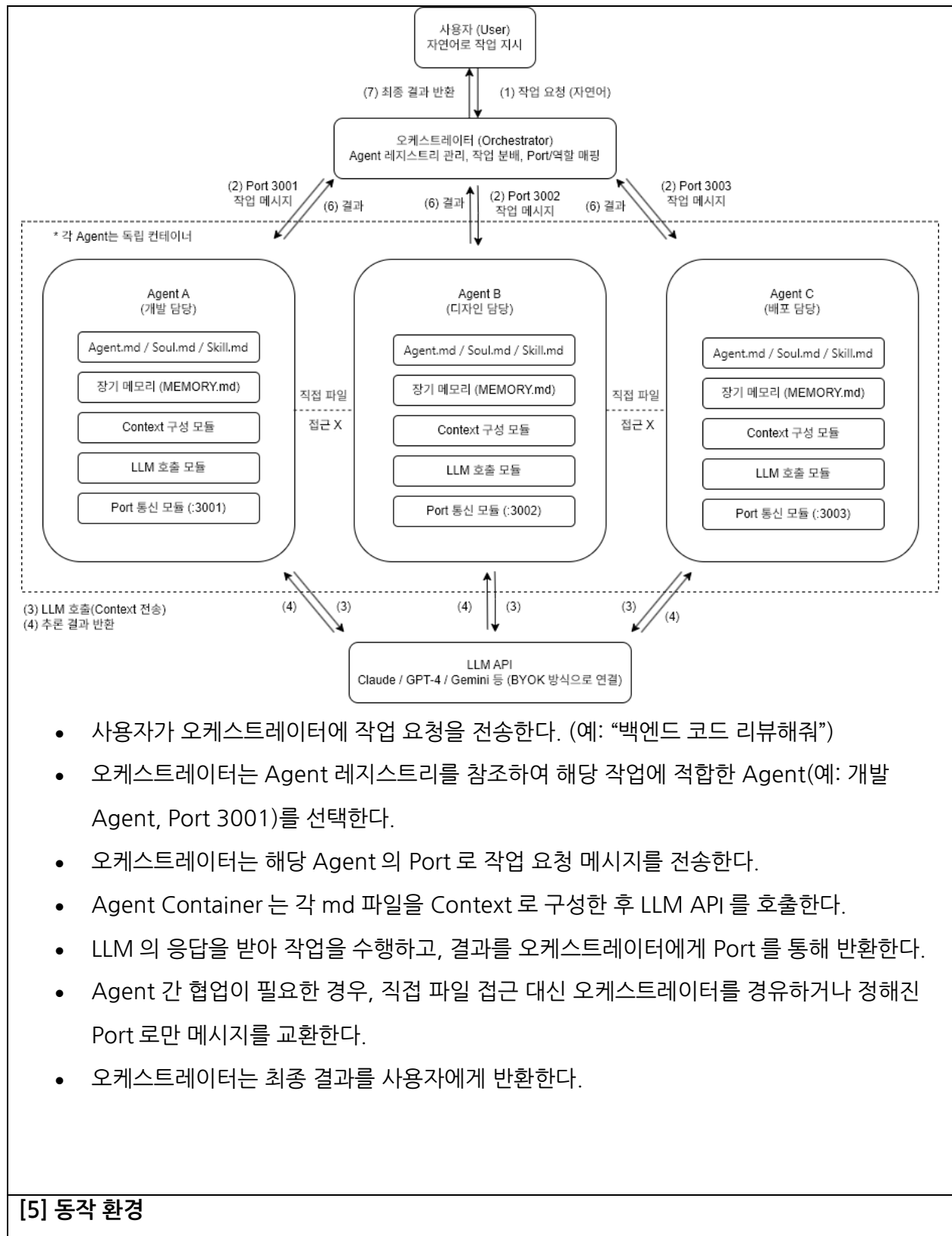
답변:

- **구성 요소**

- 1) 사용자 (User): 시스템에 작업을 지시하고 최종 결과를 전달받는 주체. 오케스트레이터에 자연어 명령을 전송한다.
- 2) 오케스트레이터 (Orchestrator): 사용자 요청을 분석하여 적절한 Agent 에게 작업을 분배하는 중앙 제어 모듈. Agent 등록 정보(Port 번호, 역할)를 관리한다.
- 3) Agent Container (Agent Container): 각 Agent 가 격리된 Docker Container 내에서 동작한다. Agent.md, Skill.md, Soul.md 를 컨텍스트로 주입받아 역할이 정의되며 장기 메모리는 Container 내부의 Text 파일로 관리된다.
- 4) Port 기반 통신 채널: Agent 간 직접 파일 접근은 불가하며 사전에 정의된 Port 를 통해서만 메시지를 교환할 수 있게 만든다.
- 5) LLM API: 각 Agent Container 가 외부 LLM(Claude, GPT 등)을 호출하여 추론 결과를 받아오는 End-point.

- **동작 흐름**

시스템의 주요 동작 흐름은 다음과 같다.



최종 결과물이 클라우드, PC/노트북, 모바일 기기(스마트폰 등), 라즈베리파이 등 어느 플랫폼에서 동작하는 지에 대한 설명도 추가하세요.

답변:

핵심 동작 단위는 Docker Container 이므로, Docker 가 설치된 환경이라면 플랫폼에 관계없이 동작 가능하도록 설계한다. 주요 타깃 플랫폼 및 환경은 다음과 같다.

- PC/노트북: 주요 개발 및 테스트 환경. Docker Engine 을 사용하여 Agent Container 를 실행한다. 소수의 Agent(3~5 개)를 운용하는 개인 개발 환경에 적합하다.
- 클라우드 환경 (AWS / GCP / Azure): 다수의 Agent 를 동시에 운용하거나 24 시간 상시 운용이 필요한 경우 클라우드 VM 또는 Container 서비스(AWS ECS, GCP Cloud Run 등)로 확장 가능한 구조를 갖추며 클라우드 환경은 본 수업의 주제와도 직결되는 만큼 클라우드 배포까지 확장해볼 수 있다.
- 모바일 및 라즈베리파이: 현재 지원 범위에 포함하지 않는다. Agent 하나당 Container 하나를 할당하는 구조 특성상 일정 수준 이상의 컴퓨팅 자원이 요구되기 때문이다.

끝! 수고하셨습니다 ☺